Beyond $1 + \sqrt{2}$ for Precedence-Constrained Scheduling of Unit Jobs

Abstract

We study the problem of minimizing total weighted completion time when unit-time jobs with precedence constraints are scheduled on identical parallel machines. The number of machines is part of the input. We give a deterministic polynomial-time approximation algorithm with ratio $1 + \sqrt{2} - 0.0288468 < 2.385367$. The algorithm adapts Li's common-quantile rounding framework for a time-indexed linear program. Our analysis relates the loss in the scalar inequality underlying the $1 + \sqrt{2}$ bound to progress forced by a band of jobs with nearby priorities. A geometric estimate charges both failures to enter this band and movement of the target quantile, and a signed-moment identity couples the resulting losses. We reduce the verification of the chosen parameters to finitely many exact quadratic inequalities. The algorithm schedules only the original jobs and is derandomized by enumerating polynomially many priority orders.

1 Introduction

Suppose a collection of tasks must be processed on several identical machines. Some tasks depend on others: for example, a computation may require data produced by an earlier computation. Each task takes one unit of time, a machine can process at most one task at a time, and a task can start only after all tasks on which it depends have finished. Finishing some tasks early is more valuable than finishing others early. We express this preference by giving each task a nonnegative weight and minimizing the sum of its weight times its completion time. Even with equal processing times, a decision to run one available task can delay other tasks and the work that depends on them. The difficulty is to balance these priorities while respecting both machine capacity and precedence.

Formally, an instance consists of n jobs numbered $1, \ldots, n$, m identical machines, nonnegative rational weights w_j, and a directed acyclic graph whose arcs specify precedence constraints. An arc is a directed edge, and acyclic means that the graph has no directed cycle. Both n and m are positive integers. Every job starts at or after time zero and occupies one machine for one unit of time without interruption. If there is an arc from i to j, job j can start only after job i completes. We seek a feasible schedule minimizing $\sum_{j=1}^{n} w_j \cdot C_j$, where C_j is the completion time of job j. In standard scheduling notation, this is $P \mid \text{prec}, p_j = 1 \mid \sum_{j=1}^{n} w_j C_j$. The number of machines is part of the input, and we impose no restriction on the precedence graph. Here P denotes identical parallel machines, prec denotes the precedence constraints, and p_j is the processing time of job j.

An *approximation algorithm with ratio* ρ always returns a feasible schedule of cost at most ρ times the optimum. Our main result is the following. Its proof appears in Theorem 12.1.

Theorem 1.1 (Main result)**.** *There is a deterministic polynomial-time* $(1 + \sqrt{2} - 0.0288468)$*-approximation for this problem. The ratio is approximately* 2.3853667624*.*

2 Related work

Hall, Schulz, Shmoys, and Wein gave a $(3 - 1/m)$-approximation for unit jobs on m machines and a 2-approximation for one machine with arbitrary processing times [1, Theorem 2.3 and page 530]. Munier, Queyranne, and Schulz gave a 4-approximation with arbitrary processing times even when precedence arcs impose nonnegative additional waiting times [2]. For arbitrary processing times and every fixed $\varepsilon > 0$, Li improved the ratio to $2 + 2\ln 2 + \varepsilon$ [3, Theorem 1.1 of the full version]. Jäger obtained the same ratio when jobs have release dates and precedence arcs can impose nonnegative waiting times [4]. A release date is a time before which a job cannot start.

Earlier scheduling algorithms used a preemptive or fractional schedule to guide the order of jobs [5, 1]. One such priority is an *α-point*, the time by which a fraction α of a job has been processed in the guiding schedule. Randomized choices of these points have also been used in approximation algorithms [6, 7].

For unit jobs, Li gave a polynomial-time $(1 + \sqrt{2})$-approximation with an arbitrary number of machines [3, Theorem 1.2 and Section 3 of the full version]. We use the full version's numbering for Li's 2017 paper throughout. The journal version is [8]. The analysis uses a time-indexed linear programming relaxation, in which a job's completion time is represented fractionally, and a common random quantile to set the priorities of jobs. For every fixed $\varepsilon > 0$, a subsequent result of Li gives a $(1 + \sqrt{2} + \varepsilon)$-approximation in nearly-linear time, meaning linear in the number of jobs and input precedence arcs up to factors polylogarithmic in the input size [9, Theorem 1.5]. These guarantees apply when the machine count is part of the input.

Several related results concern different machine or job models. On one machine with arbitrary processing times, release dates, and precedence constraints, Sitters and Yang gave a $(2 + \varepsilon)$-approximation for every fixed $\varepsilon > 0$ [10]. Jäger and Warode gave a 2-approximation for one machine and a 3-approximation for identical parallel machines in the preemptive setting [11]. Preemption allows a job to be interrupted and resumed later. Their parallel-machine result applies to arbitrary processing times and runs in strongly polynomial time.

For *related machines*, each machine has a speed and the time needed to process a job is its processing requirement divided by that speed. For arbitrary processing times and precedence constraints, Chudak and Shmoys gave an $O(\log m)$-approximation for related machines [12]. Li improved this ratio to $O(\log m / \log\log m)$ [3, Theorem 1.3 of the full version]. Chekuri, Motwani, Natarajan, and Stein used single-machine relaxations to design approximation algorithms for parallel machines and studied special precedence structures, including trees and series-parallel orders [6]. On a single machine, Lawler gave a polynomial-time exact algorithm for series-parallel orders [13]. These orders can be built from individual jobs by repeatedly placing two disjoint orders in sequence or leaving them unordered with respect to each other.

Hoogeveen, Schuurman, and Woeginger showed that there is no polynomial-time approximation scheme for unit jobs with precedence constraints and an input-dependent number of identical machines, unless P = NP, even when all weights are equal [14]. Such a scheme would provide a $(1 + \varepsilon)$-approximation for every fixed $\varepsilon > 0$.

For every fixed number of machines and fixed $\varepsilon > 0$, Garg, Kulkarni, and Li give a polynomial-time $(2 + \varepsilon)$-approximation using a stronger relaxation [15, Theorem 1.1]. Its polynomial-time guarantee is for fixed m and does not yield a polynomial-time algorithm uniformly over input-dependent m. Under the strengthened Unique Games hypothesis introduced by Bansal and Khot [16], approximation within $2 - \varepsilon$ is hard even for one machine [15, Introduction]. Roughly, this hypothesis concerns a regular bipartite graph whose edges impose permutation constraints on vertex labels. Its vertices are divided into two sides, every edge joins opposite sides, and every vertex has the same number of incident edges. Such a constraint requires one endpoint's label to be a prescribed permutation of the other endpoint's label. It asserts hardness of distinguishing instances with a labeling satisfying every constraint between two nearly complete vertex subsets from instances in which every labeling satisfies few constraints, while every subset of a specified

small fraction of one side has almost all vertices on the other side as neighbors. The precise quantified statement is given in [17, Hypothesis 1]. The scheduling hardness is conditional on this strengthened hypothesis.

3 Our contribution and proof overview

Our starting point is Li's linear-programming and common-quantile framework [3, Section 3]. The algorithm in this paper uses a continuous interpolation of the fractional completion distributions and a machine-driven priority schedule. These adapt Li's framework with a different choice of rounding rule. For a chosen quantile, the priority of each job is the time at which its fractional completion distribution reaches that quantile. The same quantile is used for every job. The precedence constraints in the linear program make each predecessor's priority earlier than its successor's. At each time slot, the algorithm schedules the available jobs of earliest priority, up to the number of machines.

The basic analysis separates the time until a particular job finishes into full and underfull slots after restricting the schedule to jobs with no later priority than the target. To count these jobs at a chosen quantile, compare each job's fractional completion by the target's fractional time with that quantile. As the quantile varies, this turns the expected prefix size into an area problem. A scalar area inequality controls this quantity and yields the $1+\sqrt{2}$ baseline. We prove a version of Li's area inequality and record additional identities for its excess [3, Claim 3.3]. Li's analysis already exploits the fact that the workload and chain bounds cannot simultaneously attain their separate worst cases [3, Section 3]. We develop a stronger coupling for this framework through priority-band progress, geometric excess, and a signed moment. Li identified an extremal scalar profile for the baseline bound [3, Section 3]. In our analysis this profile places its job inside the priority band for almost every sufficiently large quantile, while the other equality case has zero pool weight. The proof makes this connection quantitative. Specifically, the additional analysis in this manuscript uses the following ingredients.

- A priority-band lemma relates idle slots to the number of shallow jobs (jobs with short prerequisite chains) whose priorities lie in a specified interval. Informally, scheduling these jobs forces progress on earlier-priority predecessors of the target. A weighted fractional pool makes this statement useful for the linear program. We construct the pool so that it automatically has the required size.
- A geometric estimate controls, together, the jobs that miss the priority band and the change in the target's quantile over the band. It charges these losses to the excess in the scalar area inequality.
- A signed-moment identity couples the charges across jobs instead of maximizing each contribution independently. This gives a sufficient parameter certificate for a strict improvement over $1+\sqrt{2}$. The final choice is checked using exact inequalities rather than a numerical search alone.

The analysis also uses virtual zero-weight jobs to fill unused fractional capacity. These jobs are a proof device and are never scheduled by the algorithm. We prove the transfer needed to apply the padded analysis to the original instance. Finally, we enumerate the intervals on which all original jobs have the same priority order. Choosing the least costly resulting schedule removes randomness in polynomial time.

The fractional relaxation and quantile notation are developed in Section 4. Section 5 defines the schedule. The scalar inequality, priority-band estimate, and pool construction follow, after which the geometric and signed-moment bounds are assembled into a symbolic approximation certificate. The main text gives the deterministic implementation and resulting guarantee. The appendix verifies the parameter choice.

4 Preliminaries: a fractional reference schedule

We begin with the time-indexed relaxation underlying Li's common-quantile approach [3, Sections 2.1 and 3]. We spell out the relaxation and the interpolation used in this paper, since the latter also fixes the units and boundary conventions used in the proof.

Let $\mathcal{O} = \{1, \dots, n\}$ be the set of original jobs. Write $i \to j$ when the input graph contains an arc from i to j, and call i a predecessor of j when $i \neq j$ and there is a directed path from i to j. For a finite set of jobs, vertical bars denote its cardinality. For a measurable subset of the line or plane, they denote its length or area, respectively (Lebesgue measure). The *depth* a_j is the largest number of jobs on a directed path ending at j, including j. A set of jobs is *predecessor closed* if it contains every predecessor of each of its members. A *chain* is a sequence of jobs ordered by the precedence relation, and the *height* of a set of jobs is the largest number of its jobs in a chain (zero for the empty set). Every schedule satisfies $C_j \geq a_j$: the jobs on such a path must execute one after another. The depths can be computed in polynomial time by repeatedly processing a job whose predecessors have been processed. For an integer $s \geq 1$, the *slot* s is the interval $(s-1, s]$. If $m \geq n$, schedule each job j in slot a_j. This is feasible and achieves all these lower bounds. Thus we may assume $1 \leq m < n$ below.

Lemma 4.1 (A finite integer horizon)**.** *There is an optimal schedule in which every job completes at an integer time between* 1 *and* n*.*

Proof. Consider any feasible schedule and order the jobs assigned to each machine by their execution times. Add an arc between consecutive jobs in each machine order. Along each added or original arc, the second job starts no earlier than the first job's completion. Completion times therefore increase strictly along arcs, so the augmented graph is acyclic. Let a'_j be the number of vertices on a longest augmented path ending at j. Execute j in slot a'_j. The added arcs ensure that jobs assigned to the same machine do not overlap, and the original arcs ensure that precedence is respected. Every path has at most n jobs, so $a'_j \in \{1, \dots, n\}$. Along each augmented path ending at j, unit processing and the order constraints imply that the original completion time of j is at least the length of the path. Therefore the new completion time a'_j is no larger than the original one. Since weights are nonnegative, the transformed schedule costs no more. There are only finitely many integer-slot schedules within this horizon, so one of minimum cost among them is optimal even when compared with arbitrary feasible schedules. □

For each job j and slot $s \in \{1, \dots, n\}$, the variable $x_{j,s}$ is its fractional completion mass in slot s. We use the following linear program (LP). A sum with an upper limit below its lower limit is zero.

$$
\begin{aligned}
\min \quad & \sum_{j \in \mathcal{O}} w_j \cdot \sum_{s=1}^{n} s \cdot x_{j,s}, && \\
& \sum_{s=1}^{n} x_{j,s} = 1 && \text{for every } j, \\
& \sum_{j \in \mathcal{O}} x_{j,s} \leq m && \text{for every } s, \\
& \sum_{s'=1}^{s} x_{j,s'} \leq \sum_{s'=1}^{s-1} x_{i,s'} && \text{for every } i \to j \text{ and every } s, \\
& x_{j,s} \geq 0. &&
\end{aligned}
$$

The precedence constraint says that mass completing j by slot s must be matched by mass completing each job with an arc to j by slot $s-1$. For a feasible solution, put $C_j^{\mathrm{LP}} = \sum_{s=1}^{n} s \cdot x_{j,s}$.

The depth-support observation follows the corresponding argument in Li's analysis [3, Claim 3.1].

Lemma 4.2 (Lower bound and depth support)**.** *Every feasible integer-slot schedule completing by time n gives a feasible LP solution with the same objective. Moreover, every feasible LP solution satisfies $x_{j,s} = 0$ for $s < a_j$.*

Proof. Put unit mass in the completion slot of each job. Assignment, capacity, and precedence are then immediate, and the objective agrees with the schedule. Hence the LP optimum is a lower bound on the scheduling optimum. For the support claim, take a path $i_1 \to \cdots \to i_{a_j} = j$. Repeatedly applying the cumulative precedence inequalities gives, for $s < a_j$,

$$\sum_{s'=1}^{s} x_{j,s'} \le \sum_{s'=1}^{s-(a_j-1)} x_{i_1,s'} = 0.$$

Nonnegativity now gives the claim. The same repeated application shows how the constraints on arcs propagate along any path. □

The analysis below applies to any feasible LP solution. The algorithm uses an optimal solution, so its fractional objective is a lower bound on the optimal scheduling cost.

4.1 Virtual capacity padding

It is useful to make the LP capacity exactly full in the analysis. Summing the assignment constraints gives

$$\sum_{s=1}^{n} \left(m - \sum_{j \in \mathcal{O}} x_{j,s} \right) = n \cdot (m-1).$$

If $m > 1$, introduce a set $\mathcal{D}$ of $n \cdot (m-1)$ independent *virtual jobs*, each of weight zero and with no precedence arcs. Give each virtual job the fractional distribution

$$x_{i,s} = \frac{m - \sum_{j \in \mathcal{O}} x_{j,s}}{n \cdot (m-1)} \qquad (i \in \mathcal{D},\ 1 \le s \le n).$$

If $m = 1$, let $\mathcal{D}$ be empty.

Lemma 4.3 (Exact fractional capacity)**.** *The virtual distributions are valid, and after padding every slot has exactly m units of completion mass. The original-job distributions and the weighted LP objective are unchanged.*

Proof. By the capacity constraints, every numerator is nonnegative. For $m > 1$, the displayed sum shows that each virtual job's masses sum to one and their total mass in each slot fills the unused capacity. For $m = 1$, the nonnegative unused capacities sum to zero, so every slot was already full. Virtual jobs have zero weight and do not alter any original distribution. □

Let $\mathcal{J} = \mathcal{O} \cup \mathcal{D}$. Thus $|\mathcal{J}| = m \cdot n$ and

$$\sum_{i \in \mathcal{J}} x_{i,s} = m \qquad (1 \le s \le n).$$

The algorithm will schedule only the original jobs. The later virtual-pool transfer proposition explains why padded quantities may nevertheless be used to bound that schedule.

4.2 Continuous completion quantiles

Spread mass $x_{i,s}$ uniformly over the slot $(s-1, s]$. Its *cumulative distribution function* (CDF) is, for $t \in [s-1, s]$,

$$F_i(t) = \sum_{s'=1}^{s-1} x_{i,s'} + (t - s + 1) \cdot x_{i,s}.$$

Extend F_i by zero for $t \le 0$ and by one for $t \ge n$. Each F_i is continuous and nondecreasing. Exact slot capacity gives the identity

$$\sum_{i \in \mathcal{J}} F_i(t) = m \cdot t \qquad (0 \le t \le n). \tag{1}$$

Indeed, all earlier slots contribute m each and the current slot contributes $m \cdot (t - s + 1)$.

Lemma 4.4 (Continuous precedence)**.** *For every arc $i \to j$ and every real t,*

$$F_j(t) \le F_i(t-1). \tag{2}$$

Proof. At integer times inside the horizon, this is the cumulative LP precedence constraint. At the remaining integer times it follows from the extensions and the constraint at the horizon. Between successive integers, both sides are affine, meaning linear plus a constant, so the difference is affine as well. It is nonpositive at both endpoints and therefore throughout the interval. □

For $0 < \alpha < 1$ define the *generalized inverse*, or *quantile*,

$$M_i(\alpha) := \inf\{t \in [0, n] : F_i(t) \ge \alpha\}.$$

At the endpoints, extend M_i by its one-sided limits:

$$M_i(0) = \lim_{\alpha \downarrow 0} M_i(\alpha), \qquad M_i(1) = \lim_{\alpha \uparrow 1} M_i(\alpha).$$

These limits exist because M_i is nondecreasing and bounded. The extension at 0 is not obtained by substituting $\alpha = 0$ into the infimum above. Intuitively, $M_i(\alpha)$ is the interpolated time by which an α-fraction of job i has completed. Although F_i is continuous, M_i can jump when F_i has a flat interval.

For a statement A, write $\mathbf{1}\{A\}$ for its indicator: one when A holds and zero otherwise.

Lemma 4.5 (Quantile separation and mean)**.** *For every $0 < \alpha < 1$, continuity gives $F_i(M_i(\alpha)) = \alpha$. For each arc $i \to j$,*

$$M_i(\alpha) + 1 \le M_j(\alpha). \tag{3}$$

Also $M_j(0) \ge a_j - 1$, and

$$\int_0^1 M_j(\alpha)\, d\alpha = \sum_{s=1}^{n} (s - \tfrac{1}{2}) \cdot x_{j,s} = C_j^{\mathrm{LP}} - \tfrac{1}{2}. \tag{4}$$

Proof. Continuity of F_i and the definition of its generalized inverse give $F_i(M_i(\alpha)) = \alpha$. Applying (2) at $t = M_j(\alpha)$ gives $F_i(M_j(\alpha) - 1) \ge \alpha$ and hence the separation inequality. Depth support puts all of j's mass in slots a_j and later, so $M_j(\alpha) \ge a_j - 1$.

For the mean, interchange the order of integration in $\int_0^1 \int_0^n \mathbf{1}\{M_j(\alpha) > t\}\, dt\, d\alpha$. For fixed t, the measure of α with $M_j(\alpha) > t$ is $1 - F_j(t)$, giving

$$\int_0^1 M_j(\alpha)\, d\alpha = \int_0^n (1 - F_j(t))\, dt.$$

This is the mean of the interpolated distribution. The portion in slot s has mass $x_{j,s}$ and mean $s - \frac{1}{2}$, which proves the identity. The half-unit correction will be kept throughout the analysis. The LP objective itself still uses integer completion slots. □

5 Priority scheduling and prefix estimates

Choose a single α uniformly from $(0,1)$ and assign each original job i the key $M_i(\alpha)$. Write $\mathbb{E}[X]$ for the expectation of X over this choice of α. Resolve equal keys by increasing job number. At the beginning of each integer slot, schedule the $\min\{m, \text{number of ready jobs}\}$ ready original jobs with smallest keys, using this tie order. A job is *ready* if it has not yet been scheduled and all its predecessors have completed. Every scheduled job occupies the whole slot. A rule that schedules as many ready jobs as possible in each slot is called *non-delay*. This machine-driven list rule is feasible and finishes in at most n slots: whenever original jobs remain, a minimal unfinished original job is ready. Virtual jobs are used only in the analysis and are never scheduled. The complete deterministic algorithm is summarized below. Its polynomial implementation is proved in Section 11.

We represent a feasible integer-slot schedule by its completion-time vector. Within each slot, assign jobs in increasing job order to machines in increasing order. For a set of ready jobs, first_k returns its first k members in increasing order of $M_j(\alpha)$, with ties broken by increasing job number. An isolated zero has a neighborhood containing no other zeros. In the return statement, arg min selects the lexicographically first minimizing vector, comparing vectors at their first differing coordinate. Use a fixed deterministic polynomial-time LP algorithm to choose a rational optimal solution in the step below.

Algorithm 1: Deterministic common-quantile scheduling

Input: $\mathcal{O}$, the precedence relation, $(w_j)_{j\in\mathcal{O}}$, m
Output: A feasible completion-time vector
if $m \geq n$ **then**
  return $(a_j)_{j\in\mathcal{O}}$
Compute an optimal LP solution $(x_{i,s})_{i\in\mathcal{O},\,1\leq s\leq n}$
$\mathcal{C} \leftarrow \{0,1\} \cup \{F_i(s) : i \in \mathcal{O},\ 1 \leq s \leq n\}$
$\mathcal{C} \leftarrow \mathcal{C} \cup \{\alpha \in (0,1) : \alpha \text{ is an isolated zero of } M_i - M_k \text{ for some distinct } i,k \in \mathcal{O}\}$
$\mathcal{S} \leftarrow \varnothing$
foreach *consecutive* $\alpha_- < \alpha_+$ *in* $\mathcal{C}$ **do**
  $\alpha \leftarrow (\alpha_- + \alpha_+)/2$
  $\mathcal{W} \leftarrow \mathcal{O}$
  $s \leftarrow 1$
  while $\mathcal{W} \neq \varnothing$ **do**
    $\mathcal{U} \leftarrow \{j \in \mathcal{W} : \{i \in \mathcal{O} : i \to j\} \cap \mathcal{W} = \varnothing\}$
    $\mathcal{V} \leftarrow \mathrm{first}_{\min\{m,|\mathcal{U}|\}}(\mathcal{U})$
    $(C_j)_{j\in\mathcal{V}} \leftarrow (s)_{j\in\mathcal{V}}$
    $\mathcal{W} \leftarrow \mathcal{W} \setminus \mathcal{V}$
    $s \leftarrow s + 1$
  $\mathcal{S} \leftarrow \mathcal{S} \cup \{(C_j)_{j\in\mathcal{O}}\}$
return $\displaystyle\operatorname*{arg\,min}_{(C_j)_{j\in\mathcal{O}}\in\mathcal{S}} \sum_{j\in\mathcal{O}} w_j \cdot C_j$

The completed jobs form a predecessor-closed set, so checking incoming arcs suffices to identify ready jobs. The set $\mathcal{U}$ is nonempty whenever $\mathcal{W}$ is nonempty because the precedence graph is acyclic. If two jobs' quantile functions agree throughout an interval, increasing job number resolves their tie and no extra breakpoint is needed. Section 11 proves polynomial running time and bounds the cost returned by Algorithm 1.

Fix an original target j and a parameter α. Define

$$\mathcal{J}_\alpha = \{i \in \mathcal{J} : M_i(\alpha) \leq M_j(\alpha)\}.$$

The original prefix $\mathcal{J}_\alpha \cap \mathcal{O}$ includes all original jobs tied with j, including those later in the fixed

tie order. It is predecessor closed by (3).

Lemma 5.1 (Restriction to a priority prefix)**.** *Running the priority rule only on $\mathcal{J}_\alpha \cap \mathcal{O}$ gives the same completion times for its jobs as running it on all original jobs.*

Proof. Induct on slots. The ready jobs in $\mathcal{J}_\alpha \cap \mathcal{O}$ agree in the two runs. They all have higher priority than original jobs outside $\mathcal{J}_\alpha \cap \mathcal{O}$, and the latter jobs can only fill unused places in the current slot. As processing times are one, these outside jobs reserve no capacity in subsequent slots. □

Let $\widetilde{C}_j(\alpha)$ be the target's actual completion time. A slot is *underfull* if fewer than m jobs run in it. Let $N_j(\alpha)$ count underfull slots through the target's completion in the schedule restricted to $\mathcal{J}_\alpha \cap \mathcal{O}$.

The following prefix estimate adapts Li's workload and depth argument to our priority rule [3, Lemma 3.2].

Lemma 5.2 (Workload and height bounds)**.** *For the restricted schedule,*

$$\widetilde{C}_j(\alpha) \le |\mathcal{J}_\alpha \cap \mathcal{O}|/m + N_j(\alpha), \qquad N_j(\alpha) \le a_j.$$

Proof. The first inequality counts fully occupied slots, each using m distinct original prefix jobs. For the second, consider the set of unfinished predecessors of j, including j if it is unfinished. We call this the *unfinished ancestor set* of j. It is predecessor closed within the unfinished jobs. In an underfull slot every ready prefix job executes, so all minimal jobs of this set execute and its height decreases by at least one. Its initial height is a_j, and it never increases. This includes the slot in which j finishes. □

To preserve the exact-capacity scalar identities, define the analytical *prefix correction*

$$I_j(\alpha) := N_j(\alpha) - |\mathcal{J}_\alpha \cap \mathcal{D}|/m.$$

When α is random, we omit its argument from I_j and $\widetilde{C}_j$. The prefix correction may be negative or nonintegral. It is not an underfull-slot count. Its purpose is to credit the virtual prefix work that the actual scheduler never executes. The preceding estimates give

$$\widetilde{C}_j(\alpha) \le |\mathcal{J}_\alpha|/m + I_j(\alpha), \qquad I_j(\alpha) \le a_j. \tag{5}$$

The priority-band lemma below is first proved for the actual underfull count. Proposition 7.4 then shows that its weighted conclusion applies to I_j with the full virtual pool.

Following Li's scalar reduction [3, Section 3], write $h_i(\alpha) = F_i(M_j(\alpha))$ for every original or virtual job. Each h_i is nondecreasing and measurable because F_i is continuous and M_j is nondecreasing. Then

$$|\mathcal{J}_\alpha| = \sum_{i \in \mathcal{J}} \mathbf{1}\{h_i(\alpha) \ge \alpha\}, \qquad \sum_{i \in \mathcal{J}} h_i(\alpha) = m \cdot M_j(\alpha). \tag{6}$$

The first identity follows from the quantile definition and continuity of the CDFs. The second follows from the exact capacity provided by our padding in (1). No virtual-job priority need be computed by the algorithm.

6 Scalar area and excess

Write $c = \sqrt{2} - 1$, so $c + 2 = 1 + \sqrt{2}$ and $(c+2) \cdot c = 1$. For a real number x, let $(x)_+ = \max\{x, 0\}$. The next lemma refines Li's scalar bound [3, Claim 3.3] by retaining the area above the baseline. We include the proof and the exact identities needed below.

Let $h : [0,1] \to [0,1]$ be nondecreasing and measurable. Define

$$u = h(0), \qquad \beta = |\{z \in [0,1] : h(z) \geq z\}|.$$

The *excess* and *deficit* are

$$\zeta = \int_0^1 h(z)\,dz - \frac{u^2}{2} - \beta + \frac{\beta^2}{2}, \qquad f(u,\beta) = (c+2)\cdot\left(\frac{u^2}{2} + \beta - \frac{\beta^2}{2}\right) - u - \beta.$$

Lemma 6.1 (Scalar area and excess)**.** *We have* $0 \leq u \leq \beta \leq 1$ *and*

$$\zeta \geq \int_0^1 (h(z) - \max\{u,z\})_+\,dz \geq 0.$$

The deficit $f(u,\beta)$ *is nonnegative and satisfies*

$$f(u,\beta) = \frac{c+2}{2}\cdot(u - c\cdot\beta)^2 + (1-\beta)\cdot((c+1)\cdot\beta - u) \geq 0, \tag{7}$$

$$f(u,\beta) = \frac{c+2}{2}\cdot\left[(u-c)^2 + (1-\beta)\cdot(\beta - c^2)\right]. \tag{8}$$

Also, for every $z \in [0,1]$,

$$h(z) \geq \max\{u, z - (1-\beta)\}.$$

Proof. Let $\mathcal{N} = \{z \in [0,1] : h(z) < z\}$. Its measure is $1-\beta$. Since $h \geq u$, we have $h(z) \geq z$ for $z \in [0,u]$, so $u \leq \beta$. For $z \in \mathcal{N}$, monotonicity gives $(h(z), z] \subseteq \mathcal{N}$. Thus $z - h(z) \leq |\mathcal{N} \cap [0,z]|$. The second integral in the next display counts the area of pairs $(x,z) \in \mathcal{N}^2$ with $x \leq z$. Swapping the coordinates exchanges this region with its complement in $\mathcal{N}^2$, apart from the diagonal, which has area zero. Thus the region has half the area of $\mathcal{N}^2$, and

$$\int_{\mathcal{N}} (z - h(z))\,dz \leq \int_{\mathcal{N}} |\mathcal{N} \cap [0,z]|\,dz = (1-\beta)^2/2.$$

The decomposition

$$\begin{aligned}\int_0^1 h(z)\,dz &= \underbrace{\frac{1+u^2}{2}}_{\text{baseline area}} + \underbrace{\int_0^1 (h(z) - \max\{u,z\})_+\,dz}_{\text{excess above the baseline}} \\ &\quad - \underbrace{\int_{\mathcal{N}} (z - h(z))\,dz}_{\text{shortfall below the diagonal}} \\ &\geq \frac{u^2}{2} + \beta - \frac{\beta^2}{2} + \int_0^1 (h(z) - \max\{u,z\})_+\,dz\end{aligned}$$

proves the excess statement. The two identities follow by expansion using $(c+2)\cdot c = 1$ and $c^2 + 2\cdot c = 1$. The second term in (7) is nonnegative because $u \leq \beta$ and $c + 1 > 1$. Finally, if $h(z) < z$, the interval $(h(z), z]$ has measure at most $1-\beta$, proving the last assertion. The other case and the lower bound $h \geq u$ are immediate. □

Li already identified the nonzero extremal profile below [3, Section 3]. Its connection to the priority band motivates our refinement. If $f(u,\beta) = 0$ and $\zeta = 0$, the first identity and $u \leq \beta$ give $\beta \in \{0,1\}$. If $\beta = 0$, then $u = 0$ and $h = 0$ almost everywhere. If $\beta = 1$, then $u = c$. Since $h(z) \geq \max\{c,z\}$ almost everywhere and $\zeta = 0$, the excess bound gives $h(z) = \max\{c,z\}$ almost everywhere.

7 Progress forced by a priority band

The following structural statements concern an arbitrary non-delay priority list schedule of unit jobs and its underfull-slot counts N_j. The idea is that a scheduled job with a late key cannot occupy a machine while an earlier-key job is ready but unscheduled. A sequence of slots containing late-key jobs therefore removes layers of early-key predecessors of the target, even when those slots are full. Figure 1 shows an example.

Fix a target j and a real threshold t. Let $\mathcal{I}$ be the set of predecessors of j with key at most t.

Lemma 7.1 (Early-predecessor progress)**.** *Suppose keys increase by at least one along each precedence edge. After k slots each containing a job with key greater than t, the unfinished part of $\mathcal{I}$ has height at most $(a_j - 1 - k)_+$.*

Proof. The set $\mathcal{I}$ is predecessor closed and has height at most $a_j - 1$. In each specified slot, every ready job of $\mathcal{I}$ is scheduled: otherwise it would have priority over the scheduled job with key greater than t. Consequently all minimal unfinished members of $\mathcal{I}$ are removed, and its height falls by at least one whenever it is nonempty. □

Lemma 7.2 (Counting keys on a chain)**.** *Let $t < t'$. If consecutive keys on a chain differ by at least one, the number of its jobs with keys in $(t, t']$ is at most $\lceil t' - t \rceil$.*

Proof. If there are no such jobs, the claim is immediate because $t < t'$. Otherwise, if there are k such jobs, then $k - 1 < t' - t$. Since k is an integer, the result follows. □

Fix $0 < \kappa < \alpha < 1$ and consider a non-delay priority list schedule of unit jobs on m machines. Its keys satisfy $M_r(\alpha) \geq M_i(\alpha) + 1$ on every precedence arc $i \to r$. Fix a target j of depth a_j and restrict to all jobs of key at most $M_j(\alpha)$, retaining every tie. Let $N_j(\alpha)$ be the number of underfull integer slots through the target's completion in this restricted schedule. The schedule continues until every job in the prefix has completed.

Continuity gives $F_j(M_j(\alpha - \kappa)) = \alpha - \kappa$ and $F_j(M_j(\alpha)) = \alpha$. Since M_j is nondecreasing, $M_j(\alpha - \kappa) < M_j(\alpha)$. Let $d_0 \geq 0$ and let $\mathcal{K}$ be a set of jobs of depth at most d_0 whose keys lie in $(M_j(\alpha - \kappa), M_j(\alpha)]$.

Lemma 7.3 (Priority-band bound)**.** *Then*

$$N_j(\alpha) \leq d_0 + (a_j - |\mathcal{K}|/m)_+ + M_j(\alpha) - M_j(\alpha - \kappa). \tag{9}$$

Proof. We use the height argument from Lemma 5.2: at a slot boundary, the height of the unfinished ancestor set bounds the number of subsequent underfull slots through the target completion.

Assume $\mathcal{K}$ is nonempty. Let s^* be the latest completion slot of any job in $\mathcal{K}$, and choose a job $i \in \mathcal{K}$ completing at s^*. At least $\lceil |\mathcal{K}|/m \rceil$ distinct slots contain a job of $\mathcal{K}$. Let $\mathcal{I}$ be the predecessors of j whose keys are at most $M_j(\alpha - \kappa)$. At the start of slot s^*, at least $\lceil |\mathcal{K}|/m \rceil - 1$ earlier slots containing jobs of $\mathcal{K}$ have occurred. By Lemma 7.1, the remaining height of $\mathcal{I}$ is at most $(a_j - \lceil |\mathcal{K}|/m \rceil)_+$.

Any chain in the unfinished ancestor set of j at the start of s^* splits into an initial segment with key at most $M_j(\alpha - \kappa)$ and a terminal segment with key greater than $M_j(\alpha - \kappa)$. The former has length at most $(a_j - \lceil |\mathcal{K}|/m \rceil)_+$. By Lemma 7.2, the terminal segment has at most $\lceil M_j(\alpha) - M_j(\alpha - \kappa) \rceil$ jobs. The whole remaining ancestor height is therefore at most

$$(a_j - \lceil |\mathcal{K}|/m \rceil)_+ + \lceil M_j(\alpha) - M_j(\alpha - \kappa) \rceil.$$

Before slot s^*, at most $a_i - 1$ slots are underfull. Each such slot decreases the height of the unfinished ancestor set of i, which still contains i at the start of slot s^*. From slot s^* through j's

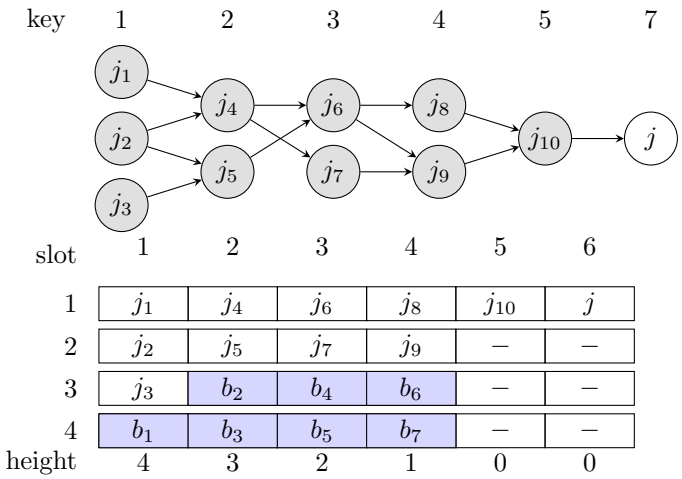


Figure 1: A priority-band example with four machines. The graph shows the predecessors of the target j and their precedence arcs. The number above each column is the priority key. The seven independent jobs $b_1, \dots, b_7$ have depth one and key 6. The jobs are numbered in the order $j_1, \dots, j_{10}, j, b_1, \dots, b_7$. With threshold 5.9, the b_i lie in the priority band $(5.9, 7]$. The table gives a non-delay priority schedule. Each of its first four slots is full and contains a band job. Its final row shows the height of the unfinished early-key predecessors after each slot, starting from height five. The band slots force this height to fall to one even though no machine is idle. Dashes denote idle machines.

completion, the number of underfull slots is at most the displayed height bound. If j has already completed, there are no such slots. Therefore, using $a_i \le \lfloor d_0 \rfloor$,

$$N_j(\alpha) \le \lfloor d_0 \rfloor + (a_j - \lceil |\mathcal{K}|/m \rceil)_+ + \lceil M_j(\alpha) - M_j(\alpha - \kappa) \rceil - 1.$$

Since $(a_j - \lceil |\mathcal{K}|/m \rceil)_+ \le (a_j - |\mathcal{K}|/m)_+$ and $\lceil M_j(\alpha) - M_j(\alpha - \kappa) \rceil - 1 \le M_j(\alpha) - M_j(\alpha - \kappa)$, this proves (9) when $\mathcal{K}$ is nonempty. If $\mathcal{K}$ is empty, the bound follows directly from $N_j(\alpha) \le a_j$, as $d_0 \ge 0$ and $M_j(\alpha) > M_j(\alpha - \kappa)$. □

For the original-only scheduler, assign each job $i \in \mathcal{J}$ a fixed weight $\theta_i \in [0, 1]$. Suppose $\sum_{i \in \mathcal{J}} \theta_i \ge m \cdot (a_j - 1)$ and each positive-weight original job has depth at most $d_0 \ge 0$.

Proposition 7.4 (Virtual-pool transfer)**.** *The prefix correction satisfies*

$$a_j - I_j(\alpha) \ge a_j - 1 - d_0 - M_j(\alpha) + M_j(\alpha - \kappa) - \frac{1}{m} \cdot \sum_{\substack{i \in \mathcal{J} \\ M_i(\alpha) \notin (M_j(\alpha-\kappa), M_j(\alpha)]}} \theta_i.$$

Proof. Let $\mathcal{K}$ be the positive-weight original jobs with keys in the band. Applying the priority-band bound gives

$$a_j - N_j(\alpha) \ge \min\left\{a_j, \frac{|\mathcal{K}|}{m}\right\} - d_0 - M_j(\alpha) + M_j(\alpha - \kappa).$$

Each virtual job in the band lies in $\mathcal{J}_\alpha \cap \mathcal{D}$. Since each weight is at most one,

$$\sum_{\substack{i \in \mathcal{J} \\ M_i(\alpha) \in (M_j(\alpha-\kappa), M_j(\alpha)]}} \theta_i \le |\mathcal{K}| + |\mathcal{J}_\alpha \cap \mathcal{D}|.$$

Thus

$$\begin{aligned}
&\min\left\{a_j, \frac{|\mathcal{K}|}{m}\right\} + \frac{|\mathcal{J}_\alpha \cap \mathcal{D}|}{m} \\
&\geq \min\left\{a_j, \frac{1}{m} \cdot \sum_{\substack{i\in\mathcal{J} \\ M_i(\alpha)\in(M_j(\alpha-\kappa),M_j(\alpha)]}} \theta_i\right\} \\
&\geq a_j - 1 - \frac{1}{m} \cdot \sum_{\substack{i\in\mathcal{J} \\ M_i(\alpha)\notin(M_j(\alpha-\kappa),M_j(\alpha)]}} \theta_i.
\end{aligned}$$

For the last step, both terms in the minimum are at least the displayed lower bound, by the assumed total pool weight and nonnegativity of the weights. Adding the virtual-prefix credit $|\mathcal{J}_\alpha \cap \mathcal{D}|/m$ to the first bound and using the definition of $I_j(\alpha)$ proves the claim. □

8 A partial pool with automatic feasibility

Fix an original target j. Extend each h_i to $[0,1]$ by setting $h_i(0)$ and $h_i(1)$ to its respective one-sided limits. These limits exist because h_i is nondecreasing and bounded. Values at the endpoints do not affect the integrals below. Let u_i, β_i, ζ_i be the resulting scalar quantities and set $f_i = f(u_i, \beta_i)$. The expected size of the full padded prefix, divided by m, is $m^{-1} \cdot \sum_{i\in\mathcal{J}} \beta_i$. Exact capacity and the quantile mean imply

$$\begin{aligned}
\sum_{i\in\mathcal{J}} u_i &= m \cdot M_j(0), \\
\frac{1}{m} \cdot \sum_{i\in\mathcal{J}} \beta_i &= (c+2) \cdot (C_j^{\mathrm{LP}} - 1/2) - M_j(0) - \frac{1}{m} \cdot \sum_{i\in\mathcal{J}} (f_i + (c+2) \cdot \zeta_i).
\end{aligned}$$

Every summand in the final sum is nonnegative by Lemma 6.1. Indeed, (6) gives $\sum_{i\in\mathcal{J}} h_i(\alpha) = m \cdot M_j(\alpha)$, including in the limit $\alpha \downarrow 0$. The definition of f_i gives $\beta_i = (c+2)\cdot(\int_0^1 h_i(\alpha)\,d\alpha - \zeta_i) - u_i - f_i$. Summing and using the quantile mean (4) proves the displayed identity for the expected prefix size. The prefix correction from (5) satisfies $I_j \leq a_j$ and $\mathbb{E}[\widetilde{C}_j] \leq \frac{1}{m} \cdot \sum_{i\in\mathcal{J}} \beta_i + \mathbb{E}[I_j]$. Therefore

$$\begin{aligned}
(c+2) \cdot C_j^{\mathrm{LP}} - \mathbb{E}[\widetilde{C}_j] \geq \frac{1}{m} \cdot \sum_{i\in\mathcal{J}} (f_i + (c+2) \cdot \zeta_i) \\
+ (M_j(0) - a_j + 1) + c/2 + \mathbb{E}[a_j - I_j].
\end{aligned} \tag{10}$$

Choose $0 < U < 1$ and define the *time cutoff*

$$d = U \cdot (a_j - 1) + \frac{1}{m} \cdot \sum_{i\in\mathcal{J}} (u_i - U)_+. \tag{11}$$

Define the pool weights by

$$\theta_i = \min\{F_i(d)/U, 1\} \qquad (i \in \mathcal{J}).$$

Lemma 8.1 (Feasibility and size of the partial pool)**.** *The cutoff* (11) *satisfies* $0 \leq d \leq M_j(0)$ *and* $\sum_{i\in\mathcal{J}} \theta_i \geq m \cdot (a_j - 1)$*. Every positive-weight original pool job has depth at most* $d + 1$*, and* $\theta_i \leq \min\{u_i/U, 1\}$ *for every job.*

Proof. The cutoff is automatically feasible: $(u - U)_+ \leq (1 - U) \cdot u$ for $0 \leq u \leq 1$, so

$$0 \leq d \leq U \cdot (a_j - 1) + (1 - U) \cdot M_j(0) \leq M_j(0).$$

These pool weights are unrelated to the objective weights. Since $d \le M_j(0)$, monotonicity and continuity give $F_i(d) \le F_i(M_j(0)) = u_i$. Exact capacity gives

$$\sum_{i\in\mathcal{J}} \theta_i = \frac{m\cdot d - \sum_{i\in\mathcal{J}} (F_i(d)-U)_+}{U} \ge \frac{m\cdot d - \sum_{i\in\mathcal{J}} (u_i-U)_+}{U} = m\cdot(a_j-1).$$

An original job of depth a_i has no interpolated mass at or before a_i-1. Thus every positive-weight original pool job has $a_i - 1 < d$, and hence depth at most $d+1$. Also $\theta_i \le \min\{u_i/U, 1\}$. □

Choose a quantile band with

$$0 < \kappa < \ell \le 1-\kappa, \qquad U \le \ell. \tag{12}$$

In particular, $\kappa < 1/2$ and $\ell < 1$. At quantile $\alpha \in [\ell, 1)$ the priority band is $(M_j(\alpha-\kappa), M_j(\alpha)]$. Define job i's integrated weighted failure to lie in this band by

$$\mathcal{F}_i = \theta_i \cdot \left|\{\alpha \in [\ell,1) : M_i(\alpha) \notin (M_j(\alpha-\kappa), M_j(\alpha)]\}\right|,$$

where the fixed pool weight θ_i is as in Lemma 8.1. The two ways to fail are disjoint: the key lies at most at the lower endpoint, or strictly above the upper endpoint. For a nondecreasing function h, write its integrated movement across the band as

$$\mathcal{M}(h) = \int_\ell^1 [h(\alpha) - h(\alpha-\kappa)]\, d\alpha.$$

Changing variables in the second integral and canceling the common part gives

$$\mathcal{M}(h) = \int_{1-\kappa}^1 h(x)\, dx - \int_{\ell-\kappa}^{\ell} h(x)\, dx.$$

For $\alpha \in [\ell, 1)$, continuity of the CDFs gives the exact event equivalences

$$\begin{aligned} M_i(\alpha) > M_j(\alpha) &\iff h_i(\alpha) < \alpha, \\ M_i(\alpha) \le M_j(\alpha-\kappa) &\iff h_i(\alpha-\kappa) \ge \alpha. \end{aligned}$$

These identities connect the subsequent scalar estimates to the actual quantile priorities.

Lemma 8.2 (Integrated progress from the partial pool)**.** *The quantities just defined satisfy*

$$\mathbb{E}[a_j - I_j] \ge (1-\ell)\cdot\big((1-U)\cdot a_j - 2 + U\big) - \frac{1}{m}\cdot\sum_{i\in\mathcal{J}} \left[(1-\ell)\cdot(u_i-U)_+ + \mathcal{F}_i + \mathcal{M}(h_i)\right]. \tag{13}$$

Proof. For $\alpha \in [\ell, 1)$, apply Proposition 7.4 with the weights above and $d_0 = d+1$. Its hypotheses hold by Lemma 8.1. Thus

$$a_j - I_j(\alpha) \ge a_j - 2 - d - \big(M_j(\alpha) - M_j(\alpha-\kappa)\big) - \frac{1}{m}\cdot \sum_{\substack{i\in\mathcal{J} \\ M_i(\alpha)\notin(M_j(\alpha-\kappa), M_j(\alpha)]}} \theta_i.$$

Integrate over $[\ell, 1]$. Since $\sum_{i\in\mathcal{J}} h_i(\alpha) = m\cdot M_j(\alpha)$,

$$\int_\ell^1 \big(M_j(\alpha) - M_j(\alpha-\kappa)\big)\, d\alpha = \frac{1}{m}\cdot\sum_{i\in\mathcal{J}} \mathcal{M}(h_i),$$

$$\int_\ell^1 \sum_{\substack{i\in\mathcal{J} \\ M_i(\alpha)\notin(M_j(\alpha-\kappa), M_j(\alpha)]}} \theta_i\, d\alpha = \sum_{i\in\mathcal{J}} \mathcal{F}_i.$$

The second identity follows from the definition of $\mathcal{F}_i$. For quantiles below ℓ, we have $a_j - I_j \ge 0$, so integration over $[\ell, 1]$ lower bounds its expectation. Substituting the definition of d gives (13). □

9 Horizontal excess and baseline cancellation

We bound lower priority-band failures and quantile movement together. Let $h : [0,1] \to [0,1]$ be nondecreasing and measurable, let $u = h(0)$, and let β and ζ be as defined in Section 6. Define the baseline $q(x) = \max\{u, x\}$. The scalar area bound gives $\int_0^1 (h(x) - q(x))_+ \, dx \le \zeta$. Use the band parameters from (12). Let $\mathcal{A}$ be the *failure set* of $\alpha \in [\ell, 1]$ with $\alpha > u$ and $h(\alpha - \kappa) \ge \alpha$. Define the *excess region* and two subregions by

$$\begin{aligned}
\mathcal{E} &= \{(x,y) : 0 \le x \le 1,\ \max\{u,x\} < y \le h(x)\},\\
\mathcal{H} &= \{(x,y) : y \in \mathcal{A},\ y - \kappa < x < y\},\\
\mathcal{R} &= \mathcal{E} \cap \{1 - \kappa < x < 1\}.
\end{aligned}$$

If $\mathcal{A}$ is nonempty, put $v = \inf \mathcal{A}$ and define the *clipped triangle*

$$\mathcal{T} = \{(x,y) : v - \kappa < x < v,\ \max\{u,x\} < y < v\}.$$

Figure 2 illustrates the construction.

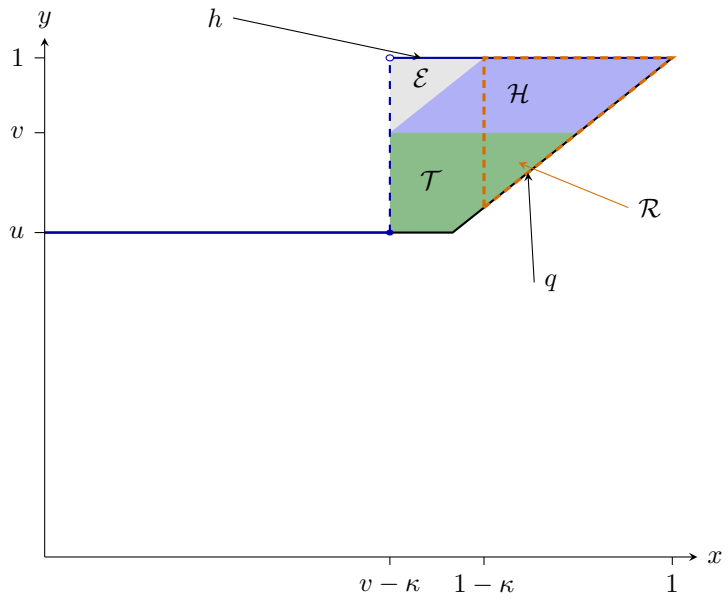


Figure 2: An example with $\kappa = 0.3$, $\ell = 0.7$, and $u = 0.65$. The function h equals 0.65 at and before its jump at 0.55 and equals 1 after the jump. The dashed segment marks the jump, and the open and filled points specify its value there. The grey background together with the blue and green regions is $\mathcal{E}$. The blue region is $\mathcal{H}$, and the green clipped triangle is $\mathcal{T}$, where the infimum of the failure levels is $v = 0.85$. Region shading ignores boundary points. The orange outline marks $\mathcal{R}$. In this example its intersections with $\mathcal{H}$ and $\mathcal{T}$ partition $\mathcal{R}$ up to boundary points, so their combined area attains $\kappa^2/2$.

Lemma 9.1 (Excess-area accounting)**.** *If $u \le \ell$, then*

$$|\mathcal{A}| + |\mathcal{R}| \le \frac{\zeta}{\kappa} + \max\left\{0, \frac{\kappa^2}{2} - \frac{\kappa^2 - (u - \ell + \kappa)_+^2}{2 \cdot \kappa}\right\}.$$

Proof. Monotonicity gives $h(x) \ge h(y - \kappa) \ge y$ for $(x,y) \in \mathcal{H}$. Since $y > \max\{u,x\}$, we have $\mathcal{H} \subseteq \mathcal{E}$, $|\mathcal{H}| = \kappa \cdot |\mathcal{A}|$, and $|\mathcal{E}| \le \zeta$. If $|\mathcal{A}| = 0$, then $|\mathcal{R}| \le \zeta \le \zeta/\kappa$.

Otherwise, $v \ge \max\{u, \ell\}$. For $v - \kappa < x < v$, choose $\alpha \in \mathcal{A}$ with $\alpha < x + \kappa$. Monotonicity gives $h(x) \ge h(\alpha - \kappa) \ge \alpha \ge v$. Consequently $\mathcal{T}$ lies in $\mathcal{E}$ and is disjoint from $\mathcal{H}$, whose points

have second coordinate at least v. Its area is

$$|\mathcal{T}| = \frac{\kappa^2 - (\kappa - v + u)_+^2}{2},$$

since the excluded part below $y = u$ is a triangle with side length $(\kappa - v + u)_+$. As $v \geq \ell \geq u$, this side length is at most $(u - \ell + \kappa)_+ \leq \kappa$. Hence

$$|\mathcal{T}| \geq \frac{\kappa^2 - (u - \ell + \kappa)_+^2}{2}.$$

The sets $\mathcal{H}$ and $\mathcal{T}$ are disjoint subsets of $\mathcal{E}$. Also $\mathcal{R} \subseteq \mathcal{E}$ and $\kappa \leq 1$. Splitting $\mathcal{R}$ according to these two sets therefore gives

$$\begin{aligned}\kappa \cdot |\mathcal{A}| + |\mathcal{T}| + \kappa \cdot |\mathcal{R}| &= |\mathcal{H}| + |\mathcal{T}| + \kappa \cdot |\mathcal{R} \setminus (\mathcal{H} \cup \mathcal{T})| \\ &\quad + \kappa \cdot |\mathcal{R} \cap (\mathcal{H} \cup \mathcal{T})| \\ &\leq |\mathcal{E}| + \kappa \cdot \underbrace{|\mathcal{R} \cap (\mathcal{H} \cup \mathcal{T})|}_{\leq \kappa^2/2} \leq \zeta + \frac{\kappa^3}{2}.\end{aligned}$$

The underbraced intersection lies in the triangle $1 - \kappa < x < y \leq 1$, which has area $\kappa^2/2$. Dividing by κ and using the lower bound on $|\mathcal{T}|$ proves the claim. □

Lemma 9.2 (Movement of the ideal baseline)**.** *If $u \leq \ell$, then*

$$\mathcal{M}(q) = \kappa \cdot (1 - \ell) - \frac{(u - \ell + \kappa)_+^2}{2}.$$

Proof. Since $q(x) = x + (u - x)_+$,

$$\mathcal{M}(q) = \kappa \cdot (1 - \ell) + \int_{1-\kappa}^{1} (u - x)_+ \, dx - \int_{\ell-\kappa}^{\ell} (u - x)_+ \, dx.$$

Because $u \leq \ell \leq 1 - \kappa$, the first integral vanishes and the second equals $(u - \ell + \kappa)_+^2/2$. □

Lemma 9.3 (Upper-baseline bound)**.** *If $u \geq \ell$, then*

$$\big|\{\alpha \in [\ell, 1] : h(\alpha - \kappa) \geq \alpha\}\big| + \mathcal{M}(h) \leq \kappa \cdot (1 - \ell) + (1 - \kappa) \cdot (u - \ell) + \kappa \cdot (1 - \beta) + \frac{\zeta}{\kappa}.$$

Proof. The failure set at levels at most u has measure $u - \ell$, since $h(\alpha - \kappa) \geq u \geq \alpha$ for $\ell \leq \alpha \leq u$. Since $\kappa \leq 1$ and $\mathcal{H} \cup \mathcal{R} \subseteq \mathcal{E}$,

$$\kappa \cdot (|\mathcal{A}| + |\mathcal{R}|) \leq |\mathcal{H} \cup \mathcal{R}| + \kappa \cdot |\mathcal{H} \cap \mathcal{R}| \leq \zeta + \kappa \cdot |\mathcal{H} \cap \mathcal{R}|.$$

Dividing by κ bounds $|\mathcal{A}| + |\mathcal{R}|$. The intersection lies above u, so

$$|\mathcal{H} \cap \mathcal{R}| \leq \int_{1-\kappa}^{1} (1 - \max\{u, x\})_+ \, dx.$$

Since $q(x) = u + (x - u)_+$ and $q(x) = u$ on $[\ell - \kappa, \ell]$,

$$\mathcal{M}(q) = \int_{1-\kappa}^{1} (x - u)_+ \, dx.$$

The integrands in the last two displays add to $1 - u$, as $u \leq 1$. Thus $|\mathcal{H} \cap \mathcal{R}| + \mathcal{M}(q) \leq \kappa \cdot (1 - u)$. Finally, the last inequality in Lemma 6.1 gives $h(x) \geq q(x) - (1 - \beta)$, and the definition of $\mathcal{R}$ gives

$$\mathcal{M}(h) \leq \mathcal{M}(q) + \kappa \cdot (1 - \beta) + |\mathcal{R}|.$$

Adding the bounds and using $(u - \ell) + \kappa \cdot (1 - u) = \kappa \cdot (1 - \ell) + (1 - \kappa) \cdot (u - \ell)$ proves the claim. □

For $0 \le u \le 1$ define

$$\Gamma(u) = \begin{cases} 0, & u \le \ell - \kappa, \\ \max\left\{ -\dfrac{(u-\ell+\kappa)^2}{2}, -\dfrac{\ell-u}{4} \right\}, & \ell - \kappa \le u \le \ell, \\ (1-\kappa)\cdot(u-\ell), & u \ge \ell. \end{cases}$$

The formulas agree at the two shared endpoints.

Corollary 9.4 (Combined failure and movement bound)**.** *We have*

$$\big|\{\alpha \in [\ell, 1] : h(\alpha - \kappa) \ge \alpha\}\big| + \mathcal{M}(h) \le \kappa \cdot (1-\ell) + \Gamma(u) + \kappa \cdot (1-\beta) + \zeta/\kappa.$$

Proof. For $u \ge \ell$, Lemma 9.3 gives the claim. For $u \le \ell$, the failure set has measure $|\mathcal{A}|$. The scalar crossing bound and the definition of $\mathcal{R}$ give

$$\mathcal{M}(h) \le \int_{1-\kappa}^{1} q(x)\,dx - \int_{\ell-\kappa}^{\ell} q(x)\,dx + \kappa \cdot (1-\beta) + |\mathcal{R}|.$$

By Lemmas 9.1 and 9.2, the failure measure plus the movement is at most

$$\kappa \cdot (1-\ell) + \kappa \cdot (1-\beta) + \frac{\zeta}{\kappa} + \max\left\{ -\frac{(u-\ell+\kappa)_+^2}{2}, \left(1 - \frac{1}{\kappa}\right) \cdot \frac{\kappa^2 - (u-\ell+\kappa)_+^2}{2} \right\}.$$

The maximum is zero if $u \le \ell - \kappa$. For $\ell - \kappa \le u \le \ell$, its second branch is

$$-(1-\kappa)\cdot(\ell-u)\cdot\left(1 - \frac{\ell-u}{2\cdot\kappa}\right) \le -\frac{\ell-u}{4},$$

because $0 \le \ell - u \le \kappa \le 1/2$. The resulting bound is $\Gamma(u)$. □

10 Joint scalar charges and the signed moment

Use the parameters in Appendix A. The scalar charges and coefficient collection below also apply to any real parameters satisfying the band conditions and inequalities stated below. The final numerical step uses the particular choice in the appendix. The appendix verifies the band conditions and the following two inequalities:

$$(1-\ell)\cdot(u-U)_+ + \Gamma(u) \le \frac{Q\cdot(c+2)}{2}\cdot(u-c)^2 + \lambda\cdot(u-c) + \nu \qquad (0 \le u \le 1) \tag{14}$$

and

$$1 + \kappa\cdot(2-\ell) \le \frac{Q\cdot(1-c)}{2} + \lambda\cdot c - \nu. \tag{15}$$

The parameter values also satisfy $Q > 0$ and $\kappa \le 1$. They satisfy $Q\cdot\kappa \ge c$ as well.

Lemma 10.1 (Charge when $\beta_i > c$)**.** *For every $i \in \mathcal{J}$ with $\beta_i > c$,*

$$(1-\ell)\cdot(u_i - U)_+ + \mathcal{F}_i + \mathcal{M}(h_i) \le (\kappa\cdot(1-\ell) + \nu)\cdot\beta_i + \lambda\cdot(u_i - c\cdot\beta_i) + Q\cdot f_i + \zeta_i/\kappa. \tag{16}$$

Proof. Since $\beta_i > c$, the deficit identity (8) gives

$$\begin{aligned} Q\cdot f_i + \lambda\cdot(u_i - c\cdot\beta_i) + \nu\cdot\beta_i &= \frac{Q\cdot(c+2)}{2}\cdot(u_i-c)^2 + \lambda\cdot(u_i-c) + \nu \\ &\quad + (1-\beta_i)\cdot\left[\frac{Q\cdot(c+2)}{2}\cdot(\beta_i - c^2) + \lambda\cdot c - \nu\right] \\ &\ge (1-\ell)\cdot(u_i-U)_+ + \Gamma(u_i) + [1+\kappa\cdot(2-\ell)]\cdot(1-\beta_i). \end{aligned}$$

The last step uses the majorant (14), the identity $(c+2)\cdot(c-c^2) = 1-c$, and (15). In particular, $\beta_i > c$ and $Q > 0$ bound the bracket below by $Q\cdot(1-c)/2 + \lambda\cdot c - \nu$. Corollary 9.4, $\theta_i \le 1$, and the upper-failure bound $1-\beta_i$ give

$$\mathcal{F}_i + \mathcal{M}(h_i) \le \kappa\cdot(1-\ell) + \Gamma(u_i) + (1+\kappa)\cdot(1-\beta_i) + \zeta_i/\kappa.$$

Writing $\kappa\cdot(1-\ell)$ as $\kappa\cdot(1-\ell)\cdot\beta_i + \kappa\cdot(1-\ell)\cdot(1-\beta_i)$ proves the claim. □

The appendix also verifies

$$\frac{1-\ell}{U} \le 1+\lambda, \qquad 1-\kappa\cdot(1-\ell)-\nu+\lambda\cdot c \le Q\cdot c. \tag{17}$$

Lemma 10.2 (Deficit bound)**.** *If $0 \le u \le \beta \le c$, then*

$$f(u,\beta) \ge c\cdot\beta.$$

Proof. The difference factors as

$$f(u,\beta) - c\cdot\beta = (\beta-u)\cdot\left[1 - \frac{c+2}{2}\cdot(\beta+u)\right].$$

Both factors are nonnegative since $u \le \beta \le c$ and $(c+2)\cdot c = 1$. □

Lemma 10.3 (Charge when $\beta_i \le c$)**.** *For every $i \in \mathcal{J}$ with $\beta_i \le c$, the bound (16) holds.*

Proof. The shifted-integral representation of $\mathcal{M}$ and $h_i - u_i \ge 0$ give

$$\mathcal{M}(h_i) \le \int_{1-\kappa}^{1} (h_i(x)-u_i)\,dx \le \int_0^1 (h_i(x)-u_i)\,dx = \zeta_i + u_i^2/2 + \beta_i - \beta_i^2/2 - u_i.$$

The total weighted failure is at most $(1-\ell)\cdot\min\{u_i/U, 1\}$, by the pool-weight bound. Thus the complete cost is at most

$$(1-\ell)\cdot[(u_i-U)_+ + \min\{u_i/U,1\}] + u_i^2/2 + \beta_i - \beta_i^2/2 - u_i + \zeta_i.$$

For $0 \le u \le 1$, the hinge terms satisfy

$$(u-U)_+ + \min\{u/U,1\} \le u/U.$$

This holds with equality for $u \le U$. For $u \ge U$, the difference between the right and left sides is $(1-U)\cdot(u/U-1) \ge 0$. Using this inequality and $u_i^2 \le \beta_i^2$, we obtain

$$\begin{aligned}
&(1-\ell)\cdot[(u_i-U)_+ + \min\{u_i/U,1\}] + u_i^2/2 + \beta_i - \beta_i^2/2 - u_i \\
&\quad - (\kappa\cdot(1-\ell)+\nu)\cdot\beta_i - \lambda\cdot(u_i - c\cdot\beta_i) \\
&\le \left[\frac{1-\ell}{U} - 1 - \lambda\right]\cdot u_i + [1-\kappa\cdot(1-\ell)-\nu+\lambda\cdot c]\cdot\beta_i \\
&\le Q\cdot c\cdot\beta_i \le Q\cdot f_i.
\end{aligned}$$

The last line uses (17) and Lemma 10.2. Since $\kappa \le 1$ and $\zeta_i \ge 0$, the complete cost is at most the claimed bound. □

10.1 Eliminating the signed moment

The following identity follows from the exact-capacity equality and the scalar gap identity in Section 8:

$$\frac{1}{m}\cdot\sum_{i\in\mathcal{J}}(u_i - c\cdot\beta_i) = (c+1)\cdot M_j(0) - C_j^{\mathrm{LP}} + \tfrac{1}{2} + \frac{c}{m}\cdot\sum_{i\in\mathcal{J}}\big(f_i + (c+2)\cdot\zeta_i\big). \tag{18}$$

Indeed, the left side equals $M_j(0) - \frac{c}{m}\cdot\sum_{i\in\mathcal{J}}\beta_i$. Substitute the identity for the sum of the β_i and use $(c+2)\cdot c = 1$.

Lemma 10.4 (Per-job bound)**.** *For every original target j and every feasible LP solution,*

$$\mathbb{E}[\widetilde{C}_j] \le (c + 2 - 0.0288468)\cdot C_j^{\mathrm{LP}}.$$

Proof. The gap estimate (10) and $I_j(\alpha) \le a_j$ imply

$$(c+2)\cdot C_j^{\mathrm{LP}} - \mathbb{E}[\widetilde{C}_j] \ge \frac{1}{m}\cdot\sum_{i\in\mathcal{J}}\big(f_i + (c+2)\cdot\zeta_i\big) + M_j(0) - a_j + 1 + \frac{c}{2}. \tag{19}$$

Apply Lemma 8.2 and, for each job, the applicable charge from Lemmas 10.1 and 10.3. Since $Q\cdot\kappa \ge c$ and $(c+2)\cdot c = 1$, we have $\zeta_i/\kappa \le Q\cdot(c+2)\cdot\zeta_i$. Summing over jobs gives

$$\begin{aligned}(c+2)\cdot C_j^{\mathrm{LP}} - \mathbb{E}[\widetilde{C}_j] \ge{}& \frac{1}{m}\cdot\sum_{i\in\mathcal{J}}\big(f_i + (c+2)\cdot\zeta_i\big) + M_j(0) - a_j + 1 + \frac{c}{2}\\ &+ a_j\cdot(1-\ell)\cdot(1-U) - (1-\ell)\cdot(2-U)\\ &- \frac{\kappa\cdot(1-\ell)+\nu}{m}\cdot\sum_{i\in\mathcal{J}}\beta_i - \frac{\lambda}{m}\cdot\sum_{i\in\mathcal{J}}(u_i - c\cdot\beta_i)\\ &- \frac{Q}{m}\cdot\sum_{i\in\mathcal{J}}\big(f_i + (c+2)\cdot\zeta_i\big).\end{aligned}$$

Substitute (18) for the signed moment and the scalar identity for $\sum_{i\in\mathcal{J}}\beta_i$. Collecting coefficients and using $(c+2)\cdot c = 1$ gives

$$\begin{aligned}(c+2)\cdot C_j^{\mathrm{LP}} - \mathbb{E}[\widetilde{C}_j] \ge{}& [(1-\ell)\cdot(1-U) + \kappa\cdot(1-\ell) + \nu - \lambda\cdot(c+1)]\cdot a_j\\ &+ [\lambda - (c+2)\cdot(\kappa\cdot(1-\ell)+\nu)]\cdot C_j^{\mathrm{LP}}\\ &+ [1 - \lambda\cdot(c+1) + \kappa\cdot(1-\ell) + \nu]\cdot(M_j(0) - a_j + 1)\\ &+ \frac{c}{2}\cdot[1 - 2\cdot(2-U)\cdot(1-\ell)/c + \kappa\cdot(1-\ell) + \nu + \lambda\cdot(c+4)]\\ &+ \frac{1 - Q + \kappa\cdot(1-\ell) + \nu - \lambda\cdot c}{m}\cdot\sum_{i\in\mathcal{J}}\big(f_i + (c+2)\cdot\zeta_i\big).\end{aligned}$$

The final three terms have nonnegative factors

$$M_j(0) - a_j + 1, \qquad c/2, \qquad \frac{1}{m}\cdot\sum_{i\in\mathcal{J}}\big(f_i + (c+2)\cdot\zeta_i\big).$$

Each coefficient of these factors is at least $1 - 3.8665485$ by Appendix A. The sum of these factors is at most the gap by (19). Since $1 - 3.8665485 < 0$, multiplying this comparison by that number reverses its direction. Therefore

$$\begin{aligned}3.8665485\cdot\big((c+2)\cdot C_j^{\mathrm{LP}} - \mathbb{E}[\widetilde{C}_j]\big) \ge{}& [(1-\ell)\cdot(1-U) + \kappa\cdot(1-\ell) + \nu - \lambda\cdot(c+1)]\cdot a_j\\ &+ [\lambda - (c+2)\cdot(\kappa\cdot(1-\ell)+\nu)]\cdot C_j^{\mathrm{LP}}.\end{aligned}$$

The appendix proves that the first bracket is positive and that

$$\lambda - (c+2)\cdot(\kappa\cdot(1-\ell)+\nu) > 0.0288468\cdot 3.8665485.$$

Dropping the nonnegative first term and dividing by the positive number completes the proof. □

11 Polynomial implementation and derandomization

The analysis has so far considered a uniformly random common quantile. We now show how to find a schedule at least as good as the expected schedule by checking only polynomially many priorities. The LP has polynomially many variables and constraints and rational coefficients. An optimal rational solution of polynomial encoding length can therefore be found in polynomial time [18]. The algorithm uses the original-job distributions only. Virtual jobs are needed for the proof, not for this computation.

Lemma 11.1 (Finitely many priority orders)**.** *For a rational LP solution of polynomial encoding length in the input size, one can construct in polynomial time a partition of* $(0, 1)$ *into* $O(n^3)$ *open intervals and their endpoints such that the original-job priority order is constant in every open interval. A rational point of polynomial encoding length can be chosen in each interval.*

Proof. For each job i and each slot s with $x_{i,s} > 0$, the inverse distribution is affine on $(F_i(s-1), F_i(s))$:

$$M_i(\alpha) = s - 1 + \frac{\alpha - F_i(s-1)}{x_{i,s}}.$$

The endpoints $F_i(s)$ partition the quantile domain into at most n such pieces for each job. For a fixed pair i, k, merge the two sets of endpoints. This gives $O(n)$ intervals, on each of which $M_i - M_k$ is affine. If this difference has one isolated zero in an interval, add it to the partition. If it is identically zero, the fixed tie order determines the comparison throughout that interval. Otherwise its sign is constant on each interval left after adding its zero, if any. The union of the endpoints and all isolated zeros therefore fixes every pairwise comparison on each resulting open interval.

There are $O(n^2)$ pairs and $O(n)$ pieces per pair, for $O(n^3)$ breakpoints. Each breakpoint is obtained by rational arithmetic from LP entries of polynomial encoding length. A midpoint between adjacent distinct breakpoints is rational and also has polynomial encoding length. Constructing these points and sorting them takes polynomial time. □

Lemma 11.2 (Deterministic selection)**.** *The algorithm can find in polynomial time an original-job schedule with cost at most the expected cost of the random-quantile schedule.*

Proof. For every interval from Lemma 11.1, run the list rule on its chosen representative, then keep the cheapest schedule. The same priority order gives the same schedule at every point of the interval. The breakpoints themselves form a finite set and hence have probability zero. Consequently the expected cost is the weighted average of the finitely many representative costs, weighted by interval length. Their minimum is no larger than that average. Each schedule is constructed in polynomial time, and there are $O(n^3)$ candidates. □

12 The approximation guarantee

The following theorem combines the per-job bound with deterministic selection.

Theorem 12.1 (Approximation guarantee)**.** *There is a deterministic polynomial-time*

$$\left(1 + \sqrt{2} - 0.0288468\right)\text{-}\textit{approximation}$$

for $P \mid \mathrm{prec}, p_j = 1 \mid \sum_{j \in \mathcal{O}} w_j C_j$*, with arbitrary precedence directed acyclic graphs, nonnegative rational weights, and an input-dependent number of identical machines. The ratio is approximately* 2.3853667624*.*

Proof. If $m \geq n$, the depth schedule returned by Algorithm 1 is optimal by Section 4. Assume $1 \leq m < n$. Apply Lemma 10.4 to an optimal LP solution, multiply by the nonnegative weights, and sum over all original jobs. The LP optimum is a lower bound on the scheduling optimum. The approximation factor is positive, and Lemma 11.2 gives a schedule whose cost is no greater than the expected cost. □

13 Conclusion

We have presented a common-quantile rounding analysis for precedence constrained unit-time scheduling and a parameter choice proving a strict improvement over the $1 + \sqrt{2}$ bound. The argument connects priority-band progress to a scalar excess inequality and then couples otherwise separate losses through a signed moment. The algorithm uses the original jobs and a polynomial number of priority orders. Virtual capacity padding serves only the analysis.

The certificate establishes a feasible parameter choice rather than an optimal choice within the analysis. Improving the scalar bounds or combining several certificates may yield a stronger guarantee.

A Parameter certificate

A.1 A family of sufficient certificates

Consider real parameters $U, \ell, \kappa, Q, \lambda, \nu$ satisfying the band conditions (12), the majorant (14), the defect condition (15), and the low-crossing conditions (17). Assume $0 < U < 1$, $Q > 0$, and $Q \cdot \kappa \geq c$. Let $0 < \delta \leq c + 1$ satisfy

$$\begin{aligned} \delta \cdot \max\big\{1,\ & Q - \kappa \cdot (1-\ell) - \nu + \lambda \cdot c, \\ & 2 \cdot (2-U) \cdot (1-\ell)/c - \kappa \cdot (1-\ell) - \nu - \lambda \cdot (c+4), \\ & \lambda \cdot (c+1) - \kappa \cdot (1-\ell) - \nu\big\} \\ & \leq \lambda - (c+2) \cdot [\kappa \cdot (1-\ell) + \nu] \\ & \quad + \min\{0, (1-\ell) \cdot (1-U) + \kappa \cdot (1-\ell) + \nu - \lambda \cdot (c+1)\}. \end{aligned} \tag{20}$$

Proposition A.1 (Parametric guarantee)**.** *For $1 \leq m < n$, every original target j, and every feasible LP solution,*

$$\mathbb{E}[\widetilde{C}_j] \leq (c + 2 - \delta) \cdot C_j^{\mathrm{LP}}.$$

Consequently, there is a deterministic polynomial-time $(c+2-\delta)$-approximation for the weighted problem.

Proof. The scalar charges and coefficient collection in the proof of Lemma 10.4 use only the assumptions above. The maximum in (20) is at least one and bounds the three coefficient deficits used in that proof. The corresponding factors are nonnegative and sum to at most the gap by (19). The same absorption therefore shows that this maximum, multiplied by $(c+2) \cdot C_j^{\mathrm{LP}} - \mathbb{E}[\widetilde{C}_j]$, is at least

$$\big[(1-\ell) \cdot (1-U) + \kappa \cdot (1-\ell) + \nu - \lambda \cdot (c+1)\big] \cdot a_j + \big[\lambda - (c+2) \cdot (\kappa \cdot (1-\ell) + \nu)\big] \cdot C_j^{\mathrm{LP}}.$$

By Lemma 4.2 and the assignment constraint,

$$C_j^{\mathrm{LP}} = \sum_{s=a_j}^{n} s \cdot x_{j,s} \geq a_j.$$

If the first coefficient is negative, replacing a_j by C_j^{LP} only decreases the expression. If it is nonnegative, dropping the first term does the same. In either case, (20) completes the per-job bound. Weighting and deterministic selection give the stated approximation when $m < n$, as in the proof of Theorem 12.1. If $m \geq n$, the depth schedule is optimal and $c + 2 - \delta \geq 1$. □

One can seek the supremum of δ subject to these conditions. Each feasible choice gives a sufficient certificate. This family fixes the upper quantile endpoint at 1 and uses the same coefficient Q in both scalar cases. We certify one feasible choice below and make no claim that it is optimal.

A.2 A feasible choice

We now verify the parameter inequalities and coefficient bounds used in Lemma 10.4. Choose

Parameter	Value	Parameter	Value
U	0.411323	ℓ	0.491938
κ	0.108423	Q	3.820348
λ	0.252884	ν	0.003462

These values give

$$\ell - \kappa = 0.383515, \qquad 1 - \ell = 0.508062.$$

In particular, $0 < \ell - \kappa < U < \ell < 1$, $0 < \kappa < 1/2$, and $1 - \ell > \kappa$. Moreover, with $c = \sqrt{2} - 1$,

$$Q \cdot \kappa - c > 2.8 \cdot 10^{-8}.$$

A.3 Checking the majorant

Using $(c+2) \cdot c = 1$ and $(c+2) \cdot c^2 = c$, define the quadratic

$$R(u) = -\frac{Q \cdot (c+2)}{2} \cdot u^2 + (Q - \lambda) \cdot u + c \cdot (\lambda - Q/2) - \nu.$$

The residual of (14) is

$$R(u) + (1 - \ell) \cdot (u - U)_+ + \Gamma(u).$$

On $[0, \ell - \kappa]$ both additional terms vanish. The derivative of R is decreasing and satisfies

$$R'(\ell - \kappa) > 0.03025, \qquad R(\ell - \kappa) < -0.0000447.$$

Thus the residual is negative on this interval.

For the remaining intervals, the following table gives the coefficients of the polynomial added to $R(u)$ for each branch of Γ. In each row, the rightmost entry is a strict upper bound on that polynomial plus $R(u)$ for all real u. The first two rows use $u \in [\ell - \kappa, U]$, the next two use $u \in [U, \ell]$, and the last uses $u \in [\ell, 1]$.

Branch	coefficient of u^2	coefficient of u	constant	upper bound
Middle quadratic	$-1/2$	$\ell - \kappa$	$-(\ell - \kappa)^2/2$	-10^{-9}
Middle linear	0	$1/4$	$-\ell/4$	-0.0228
Middle quadratic	$-1/2$	$1 - \kappa$	$-(\ell - \kappa)^2/2 - (1 - \ell) \cdot U$	$-5.82 \cdot 10^{-8}$
Middle linear	0	$5/4 - \ell$	$-\ell/4 - (1 - \ell) \cdot U$	-0.00758
Upper linear	0	$2 - \kappa - \ell$	$-(1 - \kappa) \cdot \ell - (1 - \ell) \cdot U$	$-6.89 \cdot 10^{-8}$

To check a row, write its sum with $R(u)$ as $-a \cdot u^2 + b \cdot u + d$, where $a > 0$. Completing the square shows that its maximum over the real line is $d + b^2/(4 \cdot a)$. For example, the first row has

$$a = \frac{Q \cdot (c+2) + 1}{2}, \quad b = Q - \lambda + \ell - \kappa, \quad d = c \cdot (\lambda - Q/2) - \nu - \frac{(\ell - \kappa)^2}{2}.$$

Substituting the stated parameters yields

$$4 \cdot a \cdot \left(d + \frac{b^2}{4 \cdot a} + 10^{-9} \right) < 5.51959502 - 3.90294321 \cdot \sqrt{2} < 0.$$

The last inequality follows by squaring the positive terms, since $5.51959502^2 < 2 \cdot 3.90294321^2$. Therefore the first row's maximum is less than -10^{-9}. The other rows use the same calculation. Both middle branches are bounded separately, so their maximum is bounded as well. This proves (14).

A.4 Remaining inequalities

Direct substitution gives

$$\frac{Q \cdot (1-c)}{2} + \lambda \cdot c - \nu - [1 + \kappa \cdot (2-\ell)] > \frac{1}{20}.$$

This proves (15). The two inequalities in (17) also hold with slack:

$$1 + \lambda - \frac{1-\ell}{U} > \frac{1}{100}, \qquad Q \cdot c - \big[1 - \kappa \cdot (1-\ell) - \nu + \lambda \cdot c\big] > \frac{1}{2}.$$

Finally, the coefficient of a_j in the proof of Lemma 10.4 is positive:

$$(1-\ell) \cdot (1-U) + \kappa \cdot (1-\ell) + \nu - \lambda \cdot (c+1) > 3 \cdot 10^{-8}.$$

Direct substitution also gives

$$\begin{aligned} \max\big\{1,\ & Q - \kappa \cdot (1-\ell) - \nu + \lambda \cdot c, \\ & 2 \cdot (2-U) \cdot (1-\ell)/c - \kappa \cdot (1-\ell) - \nu - \lambda \cdot (c+4), \\ & \lambda \cdot (c+1) - \kappa \cdot (1-\ell) - \nu\big\} < 3.8665485 \end{aligned} \tag{21}$$

and

$$\lambda - (c+2) \cdot [\kappa \cdot (1-\ell) + \nu] - 0.0288468 \cdot 3.8665485 > 10^{-8}.$$

These bounds complete the numerical step in Lemma 10.4. They also verify (20) for $\delta = 0.0288468$.

References

[1] Leslie A. Hall, Andreas S. Schulz, David B. Shmoys, and Joel Wein. Scheduling to Minimize Average Completion Time: Off-Line and On-Line Approximation Algorithms. *Mathematics of Operations Research*, 22(3):513–544, 1997. doi:10.1287/moor.22.3.513.

[2] Alix Munier, Maurice Queyranne, and Andreas S. Schulz. Approximation Bounds for a General Class of Precedence Constrained Parallel Machine Scheduling Problems. In *Integer Programming and Combinatorial Optimization (IPCO)*, pages 367–382, 1998. doi:10.1007/3-540-69346-7_28.

[3] Shi Li. Scheduling to Minimize Total Weighted Completion Time via Time-Indexed Linear Programming Relaxations. In *58th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 283–294, 2017. doi:10.1109/FOCS.2017.34. Full version, whose numbering is used in this paper: `https://arxiv.org/abs/1707.08039`.

[4] Sven Jäger. Approximating Total Weighted Completion Time on Identical Parallel Machines with Precedence Constraints and Release Dates. *Operations Research Letters*, 46(5):505–509, 2018. doi:10.1016/j.orl.2018.07.006.

[5] Cynthia Phillips, Clifford Stein, and Joel Wein. Minimizing Average Completion Time in the Presence of Release Dates. *Mathematical Programming*, 82:199–223, 1998. doi:10.1007/BF01585872.

[6] Chandra Chekuri, Rajeev Motwani, Balas Natarajan, and Cliff Stein. Approximation Techniques for Average Completion Time Scheduling. *SIAM Journal on Computing*, 31(1):146–166, 2001. doi:10.1137/S0097539797327180.

[7] Andreas S. Schulz and Martin Skutella. Random-Based Scheduling: New Approximations and LP Lower Bounds. In *Randomization and Approximation Techniques in Computer Science (RANDOM)*, volume 1269 of *Lecture Notes in Computer Science*, pages 119–133, 1997. doi:10.1007/3-540-63248-4_11.

[8] Shi Li. Scheduling to Minimize Total Weighted Completion Time via Time-Indexed Linear Programming Relaxations. *SIAM Journal on Computing*, 49(4):FOCS17-409–FOCS17-440, 2020. doi:10.1137/17M1156332.

[9] Shi Li. Nearly-Linear Time LP Solvers and Rounding Algorithms for Scheduling Problems. In *50th International Colloquium on Automata, Languages, and Programming (ICALP)*, volume 261 of *Leibniz International Proceedings in Informatics*, pages 86:1–86:20, 2023. doi:10.4230/LIPIcs.ICALP.2023.86.

[10] René Sitters and Liya Yang. A $(2+\varepsilon)$-Approximation for Precedence Constrained Single Machine Scheduling with Release Dates and Total Weighted Completion Time Objective. *Operations Research Letters*, 46(4):438–442, 2018. doi:10.1016/j.orl.2018.05.007.

[11] Sven Jäger and Philipp Warode. Simple Approximation Algorithms for Minimizing the Total Weighted Completion Time of Precedence-Constrained Jobs. In *2024 Symposium on Simplicity in Algorithms (SOSA)*, pages 82–96, 2024. doi:10.1137/1.9781611977936.8.

[12] Fabian A. Chudak and David B. Shmoys. Approximation Algorithms for Precedence-Constrained Scheduling Problems on Parallel Machines that Run at Different Speeds. *Journal of Algorithms*, 30(2):323–343, 1999. doi:10.1006/jagm.1998.0987.

[13] Eugene L. Lawler. Sequencing Jobs to Minimize Total Weighted Completion Time Subject to Precedence Constraints. *Annals of Discrete Mathematics*, 2:75–90, 1978. doi:10.1016/S0167-5060(08)70323-6.

[14] Han Hoogeveen, Petra Schuurman, and Gerhard J. Woeginger. Non-Approximability Results for Scheduling Problems with Minsum Criteria. *INFORMS Journal on Computing*, 13(2):157–168, 2001. doi:10.1287/ijoc.13.2.157.10520.

[15] Shashwat Garg, Janardhan Kulkarni, and Shi Li. Lift and Project Algorithms for Precedence Constrained Scheduling to Minimize Completion Time. In *Proceedings of the Thirtieth Annual ACM–SIAM Symposium on Discrete Algorithms (SODA)*, pages 1570–1584, 2019. doi:10.1137/1.9781611975482.95. Author version: `https://cse.buffalo.edu/~shil/papers/CompletionTime-SODA2019.pdf`.

[16] Nikhil Bansal and Subhash Khot. Optimal Long Code Test with One Free Bit. In *50th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 453–462, 2009. doi:10.1109/FOCS.2009.23.

[17] Ola Svensson. Hardness of Precedence Constrained Scheduling on Identical Machines. *SIAM Journal on Computing*, 40(5):1258–1274, 2011. doi:10.1137/100810502. Author version: `https://theory.epfl.ch/osven/Ola%20Svensson_publications/SICOMP11b.pdf`.

[18] Alexander Schrijver. *Theory of Linear and Integer Programming*. John Wiley & Sons, 1986. `https://ir.cwi.nl/pub/10206`.